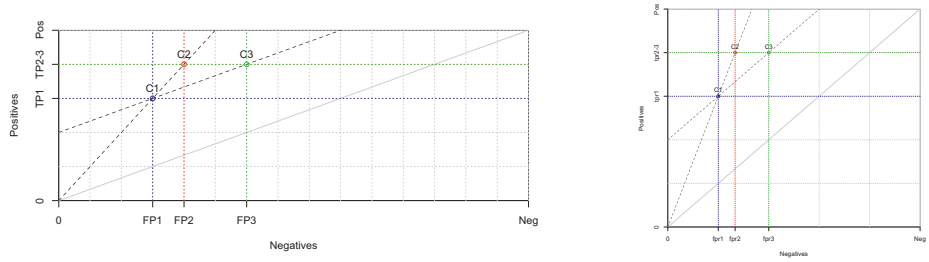

Figure 2.4. (left) In a coverage plot, accuracy isometrics have a slope of 1, and average recall isometrics are parallel to the ascending diagonal. **(right)** In the corresponding ROC plot, average recall isometrics have a slope of 1; the accuracy isometric here has a slope of 3, corresponding to the ratio of negatives to positives in the data set.

2.2 Scoring and ranking

Many classifiers compute scores on which their class predictions are based. For instance, in the Prologue we saw how SpamAssassin calculates a weighted sum from the rules that 'fire' for a particular e-mail. Such scores contain additional information that can be beneficial in a number of ways, which is why we perceive scoring as a task in its own right. Formally, a *scoring classifier* is a mapping $\hat{\mathbf{s}} : \mathcal{X} \to \mathbb{R}^k$, i.e., a mapping from the instance space to a k-vector of real numbers. The boldface notation indicates that a scoring classifier outputs a vector $\hat{\mathbf{s}}(x) = (\hat{s}_1(x), \ldots, \hat{s}_k(x))$ rather than a single number; $\hat{s}_i(x)$ is the score assigned to class C_i for instance x. This score indicates how likely it is that class label C_i applies. If we only have two classes, it usually suffices to consider the score for only one of the classes; in that case, we use $\hat{s}(x)$ to denote the score of the positive class for instance x.

Figure 2.5 demonstrates how a feature tree can be turned into a scoring tree. In order to obtain a score for each leaf, we first calculate the ratio of spam to ham, which is 1/2 for the left leaf, 2 for the middle leaf and 4 for the right leaf. Because it is often more convenient to work with an additive scale, we obtain scores by taking the logarithm of the class ratio (the base of the logarithm is not really important; here we have

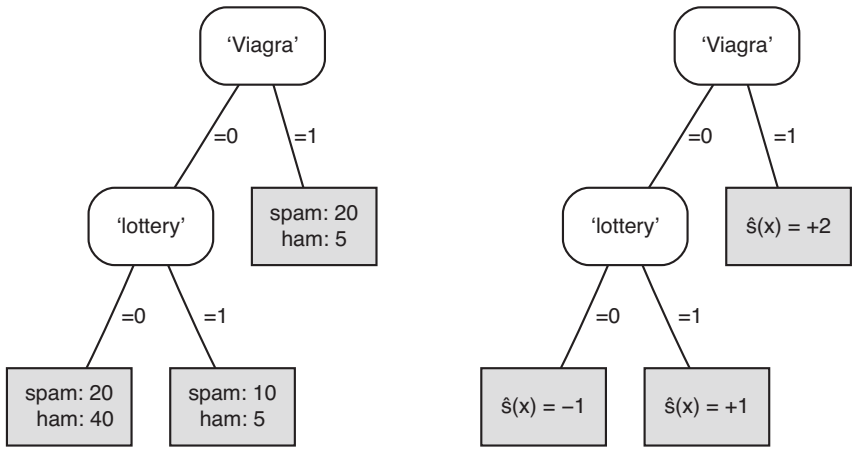

Figure 2.5. (left) A feature tree with training set class distribution in the leaves. **(right)** A scoring tree using the logarithm of the class ratio as scores; spam is taken as the positive class.

taken base-2 logarithms to get nice round numbers). Notice that the majority class decision tree corresponds to thresholding $\hat{s}(x)$ at 0: i.e., predict spam if $\hat{s}(x) > 0$ and ham otherwise.

If we take the true class $c(x)$ as $+1$ for positive examples and -1 for negative examples, then the quantity $z(x) = c(x)\hat{s}(x)$ is positive for correct predictions and negative for incorrect predictions: this quantity is called the *margin* assigned by the scoring classifier to the example.[8] We would like to reward large positive margins, and penalise large negative values. This is achieved by means of a so-called *loss function* $L : \mathbb{R} \mapsto [0, \infty)$ which maps each example's margin $z(x)$ to an associated loss $L(z(x))$. We will assume that $L(0) = 1$, which is the loss incurred by having an example on the decision boundary. We furthermore have $L(z) \geq 1$ for $z < 0$, and usually also $0 \leq L(z) < 1$ for $z > 0$ (Figure 2.6). The average loss over a test set Te is $\frac{1}{|Te|} \sum_{x \in Te} L(z(x))$.

The simplest loss function is *0–1 loss*, which is defined as $L_{01}(z) = 1$ if $z \leq 0$ and $L(z) = 0$ if $z > 0$. The average 0–1 loss is simply the proportion of misclassified test examples:

$$\frac{1}{|Te|} \sum_{x \in Te} L_{01}(z(x)) = \frac{1}{|Te|} \sum_{x \in Te} I[c(x)\hat{s}(x) \leq 0] = \frac{1}{|Te|} \sum_{x \in Te} I[c(x) \neq \hat{c}(x)] = err$$

where $\hat{c}(x) = +1$ if $\hat{s}(x) > 0$, $\hat{c}(x) = 0$ if $\hat{s}(x) = 0$, and $\hat{c}(x) = -1$ if $\hat{s}(x) < 0$. (It is sometimes more convenient to define the loss of examples on the decision boundary as $1/2$). In other words, 0–1 loss ignores the magnitude of the margins of the examples, only

[8] Remember that in Chapter 1 we talked about the margin of a classifier as the distance between the decision boundary and the nearest example. Here we use margin in a slightly more general sense: each example has a margin, not just the nearest one. This will be further explained in Section 7.3.

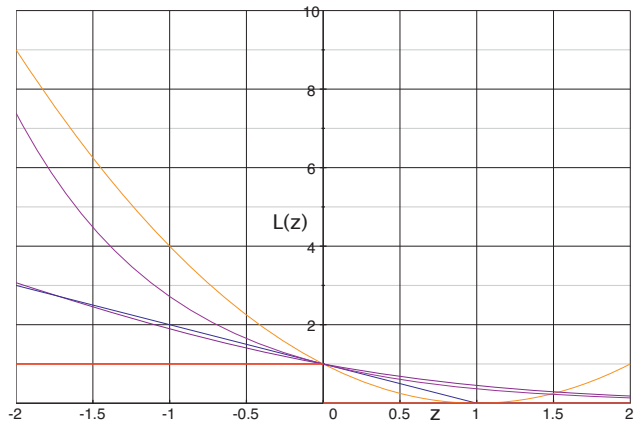

Figure 2.6. Loss functions: from bottom-left *(i)* 0–1 loss $L_{01}(z) = 1$ if $z \leq 0$, and $L_{01}(z) = 0$ if $z > 0$; *(ii)* hinge loss $L_{\mathrm{h}}(z) = (1 - z)$ if $z \leq 1$, and $L_{\mathrm{h}}(z) = 0$ if $z > 1$; *(iii)* logistic loss $L_{\log}(z) = \log_2(1 + \exp(-z))$; *(iv)* exponential loss $L_{\exp}(z) = \exp(-z)$; *(v)* squared loss $L_{\mathrm{sq}}(z) = (1 - z)^2$ (this can be set to 0 for $z > 1$, just like hinge loss).

taking their sign into account. As a result, 0–1 loss doesn't distinguish between scoring classifiers, as long as their predictions agree. This means that it isn't actually that useful as a search heuristic or objective function when learning scoring classifiers. Figure 2.6 pictures several loss functions that are used in practice. Except for 0–1 loss, they are all *convex*: linear interpolation between any two points on the curve will never result in a point below the curve. Optimising a convex function is computationally more tractable.

One loss function that will be of interest later is the *hinge loss*, which is defined as $L_{\mathrm{h}}(z) = (1 - z)$ if $z \leq 1$, and $L_{\mathrm{h}}(z) = 0$ if $z > 1$. The name of this loss function comes from the fact that the loss 'hinges' on whether an example's margin is greater than 1 or not: if so (i.e., the example is on the correct side of the decision boundary with a distance of at least 1) the example incurs zero loss; if not, the loss increases with decreasing margin. In effect, the loss function expresses that it is important to avoid examples having a margin (much) less than 1, but no additional value is placed on achieving large positive margins. This loss function is used when training a ☞*support vector machine* (Section 7.3). We will also encounter *exponential loss* later when we discuss ☞*boosting* in Section 11.2.

Assessing and visualising ranking performance

It should be kept in mind that scores are assigned by a classifier, and are not a property inherent to instances. Scores are not estimated from 'true scores' – rather, a scoring classifier has to be learned from examples in the form of instances x labelled with

classes $c(x)$, just as a classifier. (The task where we learn a function $\hat{f}$ from examples labelled with true function values $(x, f(x))$ is called ☞*regression* and is covered in Section 3.2.) Often it is more convenient to keep the order imposed by scores on a set of instances, but ignore their magnitudes – this has the advantage, for instance, of being much less sensitive to outliers. It also means that we do not have to make any assumptions about the scale on which scores are expressed: in particular, a ranker does not assume a particular score threshold for separating positives from negatives. A *ranking* is defined as a total order on a set of instances, possibly with ties.[9]

Example 2.2 (Ranking example). The scoring tree in Figure 2.5 produces the following ranking: [20+,5−][10+,5−][20+,40−]. Here, 20+ denotes a sequence of 20 positive examples, and instances in square brackets [...] are tied. By selecting a split point in the ranking we can turn the ranking into a classification. In this case there are four possibilities: (A) setting the split point before the first segment, and thus assigning all segments to the negative class; (B) assigning the first segment to the positive class, and the other two to the negative class; (C) assigning the first two segments to the positive class; and (D) assigning all segments to the positive class. In terms of actual scores, this corresponds to (A) choosing any score larger than 2 as the threshold; (B) choosing a threshold between 1 and 2; (C) setting the threshold between −1 and 1; and (D) setting it lower than −1.

Suppose x and x' are two instances such that x receives a lower score: $\hat{s}(x) < \hat{s}(x')$. Since higher scores express a stronger belief that the instance in question is positive, this would be fine except in one case: if x is an actual positive and x' is an actual negative. We will call this a *ranking error*. The total number of ranking errors can then be expressed as $\sum_{x \in Te^{\oplus}, x' \in Te^{\ominus}} I[\hat{s}(x) < \hat{s}(x')]$. Furthermore, for every positive and negative that receive the same score – a *tie* – we count half a ranking error. The maximum number of ranking errors is equal to $|Te^{\oplus}| \cdot |Te^{\ominus}| = Pos \cdot Neg$, and so the *ranking error rate* is defined as

$$rank\text{-}err = \frac{\sum_{x \in Te^{\oplus}, x' \in Te^{\ominus}} I[\hat{s}(x) < \hat{s}(x')] + \frac{1}{2} I[\hat{s}(x) = \hat{s}(x')]}{Pos \cdot Neg} \tag{2.4}$$

and analogously the *ranking accuracy*

$$rank\text{-}acc = \frac{\sum_{x \in Te^{\oplus}, x' \in Te^{\ominus}} I[\hat{s}(x) > \hat{s}(x')] + \frac{1}{2} I[\hat{s}(x) = \hat{s}(x')]}{Pos \cdot Neg} = 1 - rank\text{-}err \tag{2.5}$$

[9] A total order with ties should not be confused with a partial order (see Background 2.1 on p.51). In a total order with ties (which is really a total order on equivalence classes), any two elements are comparable, either in one direction or in both. In a partial order some elements are incomparable.

Ranking accuracy can be seen as an estimate of the probability that an arbitrary positive–negative pair is ranked correctly.

Example 2.3 (Ranking accuracy). We continue the previous example considering the scoring tree in Figure 2.5, with the left leaf covering 20 spam and 40 ham, the middle leaf 10 spam and 5 ham, and the right leaf 20 spam and 5 ham. The 5 negatives in the right leaf are scored higher than the 10 positives in the middle leaf and the 20 positives in the left leaf, resulting in $50 + 100 = 150$ ranking errors. The 5 negatives in the middle leaf are scored higher than the 20 positives in the left leaf, giving a further 100 ranking errors. In addition, the left leaf makes 800 half ranking errors (because 20 positives and 40 negatives get the same score), the middle leaf 50 and the right leaf 100. In total we have 725 ranking errors out of a possible $50 \cdot 50 = 2500$, corresponding to a ranking error rate of 29% or a ranking accuracy of 71%.

The coverage plots and ROC plots introduced in the previous section for visualising classifier performance provide an excellent tool for visualising ranking performance too. If *Pos* positives and *Neg* negatives are plotted on the vertical and horizontal axes, respectively, then each positive–negative pair occupies a unique 'cell' in this plot. If we order the positives and negatives on decreasing score, i.e., examples with higher scores are closer to the origin, then we can clearly distinguish the correctly ranked pairs at the bottom right, the ranking errors at the top left, and the ties in between (Figure 2.7). The number of cells in each area gives us the number of correctly ranked pairs, ranking errors and ties, respectively. The diagonal lines cut the ties area in half, so the area below those lines corresponds to the ranking accuracy multiplied by $Pos \cdot Neg$, and the area above corresponds to the ranking error rate times that same factor.

Concentrating on those diagonal lines gives us the piecewise linear curve shown in Figure 2.7 (right). This curve, which we will call a *coverage curve*, can be understood as follows. Each of the points marked A, B, C and D specifies the classification performance, in terms of true and false positives, achieved by the corresponding ranking split points or score thresholds from Example 2.2. To illustrate, C would be obtained by a score threshold of 0, leading to $TP2 = 20 + 10 = 30$ true positives and $FP2 = 5 + 5 = 10$ false positives. Similarly, B would be obtained by a higher threshold of 1.5, leading to $TP1 = 20$ true positives and $FP1 = 5$ false positives. Point A would result if we set the threshold unattainably high, and D if we set the threshold trivially low.

Why are these points connected by straight lines? How can we interpolate between, say, points C and D? Suppose we set the threshold exactly at -1, which is the score

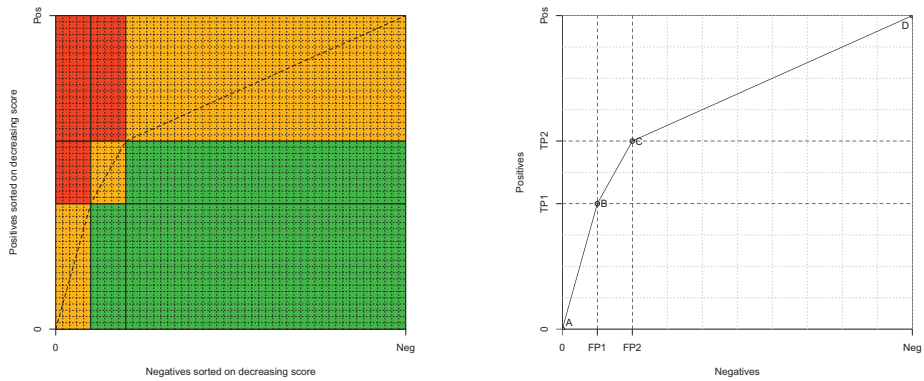

Figure 2.7. (left) Each cell in the grid denotes a unique pair of one positive and one negative example: the green cells indicate pairs that are correctly ranked by the classifier, the red cells represent ranking errors, and the orange cells are half-errors due to ties. **(right)** The coverage curve of a tree-based scoring classifier has one line segment for each leaf of the tree, and one (FP, TP) pair for each possible threshold on the score.

assigned by the left leaf of the tree. The question is now what class we predict for the 20 positives and 40 negatives that filter down to that leaf. It would seem reasonable to decide this by tossing a fair coin, leading to half of the positives receiving a positive prediction (on average) and half of them a negative one, and similar for the negatives. The total number of true positives is then $30+20/2=40$, and the number of false positives is $10+40/2=30$. In other words, we land exactly in the middle of the CD line segment. We can apply the same procedure to achieve performance half-way BC, by setting the threshold at 1 and tossing the same fair coin to obtain uniformly distributed predictions for the 10 positives and 5 negatives in the middle leaf, leading to $20+10/2=25$ true positives and $5+5/2=7.5$ false positives (of course, we cannot achieve a non-integer number of false positives in any trial, but this number represents the expected number of false positives over many trials). And what's more, by biasing the coin towards positive or negative predictions we can achieve expected performance anywhere on the line.

More generally, a coverage curve is a piecewise linear curve that rises monotonically from $(0,0)$ to (Neg, Pos) – i.e., TP and FP can never decrease if we decrease the decision threshold. Each segment of the curve corresponds to an equivalence class of the instance space partition induced by the model in question (e.g., the leaves of a feature tree). Notice that the number of segments is never more than the number of test instances. Furthermore, the slope of each segment is equal to the ratio of positive to negative test instances in that equivalence class. For instance, in our example the first segment has a slope of 4, the second segment slope 2, and the third segment slope $1/2$

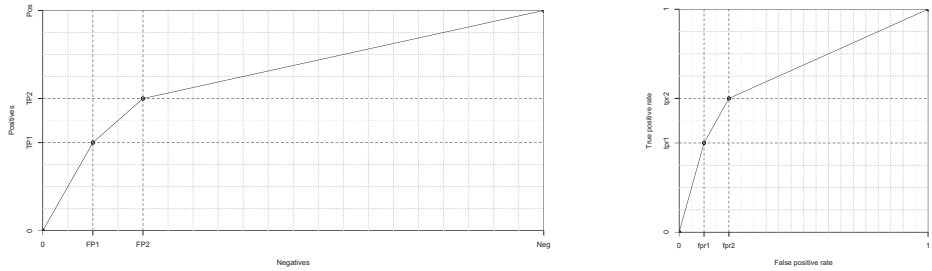

Figure 2.8. (left) A coverage curve obtained from a test set with class ratio $clr = 1/2$. **(right)** The corresponding ROC curve is the same as the one corresponding to the coverage curve in Figure 2.7 (right).

— exactly the scores assigned in each leaf of the tree! This is not true in general, since the coverage curve depends solely on the ranking induced by the scores, not on the scores themselves. However, it is not a coincidence either, as we shall see in the next section on class probability estimation.

An *ROC curve* is obtained from a coverage curve by normalising the axes to $[0,1]$. This doesn't make much of a difference in our running example, but in general coverage curves can be rectangular whereas ROC curves always occupy the unit square. One effect this has is that slopes are multiplied by $Neg/Pos = 1/clr$. Furthermore, while in a coverage plot the area under the coverage curve gives the absolute number of correctly ranked pairs, in an ROC plot *the area under the ROC curve is the ranking accuracy* as defined in Equation 2.5 on p.64. For that reason people usually write *AUC* for 'Area Under (ROC) Curve', a convention I will follow.

Example 2.4 (Class imbalance). Suppose we feed the scoring tree in Figure 2.5 on p.62 an extended test set, with an additional batch of 50 negatives. The added negatives happen to be identical to the original ones, so the net effect is that the number of negatives in each leaf doubles. As a result the coverage curve changes (because the class ratio changes), but the ROC curve stays the same (Figure 2.8). Note that the AUC stays the same as well: while the classifier makes twice as many ranking errors, there are also twice as many positive–negative pairs, so the ranking error rate doesn't change.

Let us now consider an example of a coverage curve for a grading classifier. Figure 2.9 (left) shows a linear classifier (the decision boundary is denoted B) applied to a

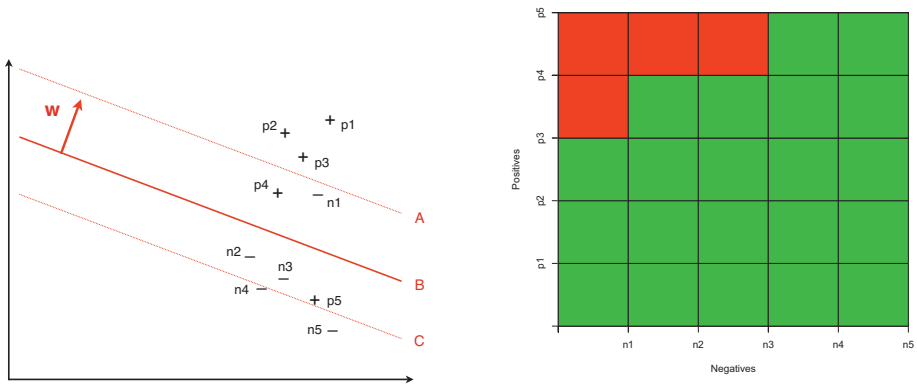

Figure 2.9. (left) A linear classifier induces a ranking by taking the signed distance to the decision boundary as the score. This ranking only depends on the orientation of the decision boundary: the three lines result in exactly the same ranking. **(right)** The grid of correctly ranked positive–negative pairs (in green) and ranking errors (in red).

small data set of five positive and five negative examples, achieving an accuracy of 0.80. We can derive a score from this linear classifier by taking the distance of an example from the decision boundary; if the example is on the negative side we take the negative distance. This means that the examples are ranked in the following order: p1 – p2 – p3 – n1 – p4 – n2 – n3 – p5 – n4 – n5. This ranking incurs four ranking errors: n1 before p4, and n1, n2 and n3 before p5. Figure 2.9 (right) visualises these four ranking errors in the top-left corner. The AUC of this ranking is $21/25 = 0.84$.

From this grid we obtain the coverage curve in Figure 2.10. Because of its stepwise character, this curve looks quite different from the coverage curves for scoring trees that we saw earlier in this section. The main reason is the absence of ties, which means that all segments in the curve are horizontal or vertical, and that there are as many segments as examples. We can generate this stepwise curve from the ranking as follows: starting in the lower left-hand corner, we go up one step if the next example in the ranking is positive, and right one step if the next example is negative. The result is a curve that goes three steps up (for p1–3), one step to the right (for n1), one step up (p4), two steps to the right (n2–3), one step up (p5), and finally two steps to the right (n4–5).

We can actually use the same procedure for grouping models if we handle ties as follows: in case of a tie between p positive examples and n negative examples, we go p steps up and *at the same time* n steps to the right. Looking back at Figure 2.7 on p.66, you will see that this is exactly what happens in the diagonal segments spanning the orange rectangles which arise as a result of the ties in the leaves of the decision tree. Thus, the principles underlying coverage and ROC curves are the same for both

grouping and grading models, but the curves themselves look quite different in each case. *Grouping model ROC curves have as many line segments as there are instance space segments in the model; grading models have one line segment for each example in the data set*. This is a concrete manifestation of something I mentioned in the Prologue: grading models have a much higher 'resolution' than grouping models; this is also called the model's *refinement*.

Notice the three points in Figure 2.10 labelled A, B and C. These points indicate the performance achieved by the decision boundaries with the same label in Figure 2.9. As an illustration, the middle boundary B misclassifies one out of five positives ($tpr = 0.80$) and one out of five negatives ($fpr = 0.80$). Boundary A doesn't misclassify any negatives, and boundary C correctly classifies all positives. In fact, while they should all have the same orientation, their exact location is not important, as long as boundary A is between p3 and n1, boundary B is between p4 and n2, and boundary C is between p5 and n4. There are good reasons why I chose exactly these three boundaries, as we shall see shortly. For the moment, observe what happens if we use all three boundaries to turn the linear model into a grouping model with four segments: the area above A, the region between A and B, the bit between B and C, and the rest below C. The result is that we no longer distinguish between n1 and p4, nor between n2–3 and p5. The ties just introduced change the coverage curve to the dotted segments in Figure 2.10. Notice that this results in a larger AUC of 0.90. Thus, *by decreasing a model's refinement we sometimes achieve better ranking performance*. Training a model is not just about amplifying significant distinctions, but also about diminishing the effect of misleading distinctions.

Turning rankers into classifiers

I mentioned previously that the main difference between rankers and scoring classifiers is that a ranker only assumes that a higher score means stronger evidence for the positive class, but otherwise makes no assumptions about the scale on which scores are expressed, or what would be a good score threshold to separate positives from negatives. We will now consider the question how to obtain such a threshold from a coverage curve or ROC curve.

The key concept is that of the accuracy isometric. Recall that in a coverage plot points of equal accuracy are connected by lines with slope 1. All we need to do, therefore, is to draw a line with slope 1 through the top-left point (which is sometimes called *ROC heaven*) and slide it down until we touch the coverage curve in one or more points. Each of those points achieves the highest accuracy possible with that model. In Figure 2.10 this method would identify points A and B as the points with highest accuracy (0.80). They achieve this in different ways: e.g., model A is more conservative on the positives.

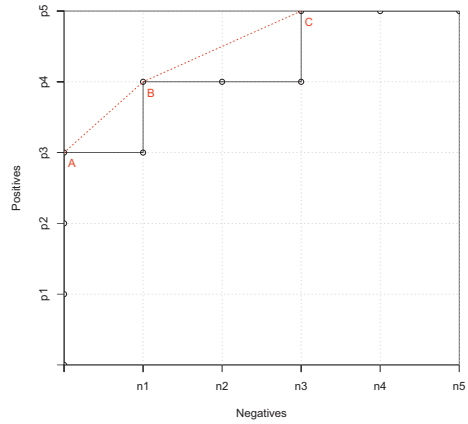

Figure 2.10. The coverage curve of the linear classifier in Figure 2.9. The points labelled A, B and C indicate the classification performance of the corresponding decision boundaries. The dotted lines indicate the improvement that can be obtained by turning the grading classifier into a grouping classifier with four segments.

A similar procedure can be followed with ROC plots, as long as you keep in mind that all slopes have to be multiplied by the reciprocal of the class ratio, $1/clr = Neg/Pos$.

Example 2.5 (Tuning your spam filter). You have carefully trained your Bayesian spam filter, and all that remains is setting the decision threshold. You select a set of six spam and four ham e-mails and collect the scores assigned by the spam filter. Sorted on decreasing score these are 0.89 (spam), 0.80 (spam), 0.74 (ham), 0.71 (spam), 0.63 (spam), 0.49 (ham), 0.42 (spam), 0.32 (spam), 0.24 (ham), and 0.13 (ham). If the class ratio of 3 spam against 2 ham is representative, you can select the optimal point on the ROC curve using an isometric with slope 2/3. As can be seen in Figure 2.11, this leads to putting the decision boundary between the sixth spam e-mail and the third ham e-mail, and we can take the average of their scores as the decision threshold (0.28).

An alternative way of finding the optimal point is to iterate over all possible split points – from before the top ranked e-mail to after the bottom one – and calculate the number of correctly classified examples at each split: 4 – 5 – 6 – 5 – 6 – 7 – 6 – 7 – 8 – 7 – 6. The maximum is achieved at the same split point, yielding an accuracy of 0.80. A useful trick to find out which accuracy an isometric in an ROC plot represents is to intersect the isometric with the descending diagonal.

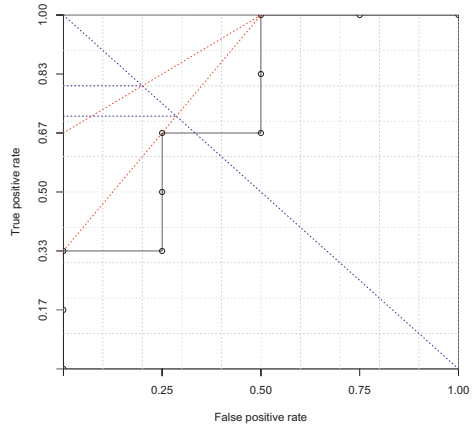

Figure 2.11. Selecting the optimal point on an ROC curve. The top dotted line is the accuracy isometric, with a slope of 2/3. The lower isometric doubles the value (or prevalence) of negatives, and allows a choice of thresholds. By intersecting the isometrics with the descending diagonal we can read off the achieved accuracy on the y-axis.

Since accuracy is a weighted average of the true positive and true negative rates, and since these are the same in a point on the descending diagonal, we can read off the corresponding accuracy value on the y-axis.

If the class distribution in the data is *not* representative, we can simply adjust the slope of the isometric. For example, if ham is in fact twice as prevalent, we use an isometric with slope 4/3. In the previous example this leads to three optimal points on the ROC curve.[10] Even if the class ratio in the data is representative, we may have other reasons to assign different weights to the classes. To illustrate, in the spam e-mail situation our spam filter may discard the false positives (ham e-mails misclassified as spam) so we may want to drive the false positive rate down by assigning a higher weight to the negatives (ham). This is often expressed as a *cost ratio* $c = c_{\text{FN}}/c_{\text{FP}}$ of the cost of false negatives in proportion to the cost of false positives, which in this case would be set to a value smaller than 1. The relevant isometrics then have a slope of $1/c$ in a coverage plot, and $1/(c \cdot clr)$ in an ROC plot. The combination of cost ratio and class ratio gives a precise context in which the classifier is deployed and is referred to as the

[10] It seems reasonable to choose the middle of these three points, leading to a threshold of 0.56. An alternative is to treat all e-mails receiving a score in the interval [0.28, 0.77] as lying on the decision boundary, and to randomly assign a class to those e-mails.

operating condition.

If the class or cost ratio is highly skewed, this procedure may result in a classifier that assigns the same class to all examples. For instance, if negatives are 1 000 times more prevalent than positives, accuracy isometrics are nearly vertical, leading to an unattainably high decision threshold and a classifier that classifies everything as negative. Conversely, if the profit of one true positive is 1 000 times the cost of a false positive, we would classify everything as positive – in fact, this is the very principle underlying spam e-mail! However, often such one-size-fits-all behaviour is unacceptable, indicating that accuracy is not the right thing to optimise here. In such cases we should use average recall isometrics instead. These run parallel to the ascending diagonal in both coverage and ROC plots, and help to achieve similar performance on both classes.

The procedure just described learns a decision threshold from labelled data by means of the ROC curve and the appropriate accuracy isometric. This procedure is often preferable over fixing a decision threshold in advance, particularly if scores are expressed on an arbitrary scale – for instance, this would provide a way to finetune the SpamAssassin decision threshold to our particular situation and preferences. Even if the scores are probabilities, as in the next section, these may not be sufficiently well estimated to warrant a fixed threshold of 0.5.